

CIVIC / PLATFORM

Product Blueprint

A-to-Z specification for the civic problem-solving platform

From a citizen's WhatsApp voice note or web report to an AI-understood, deduplicated, geographically routed, actively resolved civic problem.

Prepared for product launch planning

Pratik & The Favicons

Status: Post-SIH product direction / production blueprint

Scope: Product, UX, architecture, AI pipeline, data, security, operations, roadmap

1. Executive Summary

Civic Platform is a civic-tech system designed to turn fragmented citizen complaints into structured, actionable problems and connect those problems to the people or organizations capable of solving them.

The product accepts reports through a web interface and, in the production vision, through WhatsApp. A citizen may write a message, send a voice note, attach a photo, or provide location information. The platform interprets the input, extracts civic facts, checks media, finds related reports, consolidates duplicate complaints, routes the resulting problem using geography and responsibility rules, and tracks the problem through resolution.

The core product principle is **human agency with AI assistance**: AI interprets, structures, searches, and recommends; rules, authorized people, and organizations remain responsible for official decisions and actions.

The one-line product definition

“A civic problem operating system that converts citizen reports into verified, consolidated, routed, and trackable real-world action.”

What exists today

- A Next.js web application with citizen reporting, public problems, authentication, personal reports, officer workspace, university workspace, and admin workspace.
- Supabase-backed authentication and data access.
- AI-assisted report processing and duplicate-detection logic.
- Problem/report lifecycle and status workflows.
- University collaboration-interest flow.
- Production deployment on Vercel with environment configuration.
- The next product phase is to harden this MVP and add the production ingestion, vector, spatial, notification, reliability, and security layers.

2. The Problem We Are Solving

Civic complaints are usually fragmented. The same broken road, water leak, garbage point, unsafe streetlight, or school infrastructure issue may be reported dozens of times through different channels. Authorities may see isolated messages rather than one underlying problem.

- Citizens do not know where to report or whether somebody is acting on the issue.
- Different people describe the same problem in different words and languages.
- Voice notes and photos contain useful information but are difficult to process manually at scale.
- Duplicate complaints create noise instead of strengthening the signal.
- Responsibility depends on geography, department, jurisdiction, and expertise.
- Citizens often lack transparent status updates after submission.
- Potential solution teams—universities, NGOs, researchers, engineers, and community organizations—are rarely connected to the right civic problem.

The platform therefore treats a citizen submission as an **observation** and attempts to construct a reliable **problem entity** underneath it.

3. Who Uses the Product

Actor	Purpose	Primary capabilities
Citizen	Report and follow civic problems	Submit text/voice/photo, location, track status, receive updates
Officer / authority	Operate on assigned problems	Review, accept, update status, resolve, add evidence
Administrator	Govern the system	Manage users, assignments, reports, interests, operational controls
University / solution team	Contribute expertise	Discover relevant problems, express interest, collaborate
Platform operator	Keep product reliable	Monitor jobs, failures, notifications, security, analytics

4. Product Experience

Citizen web experience

- Landing page explains the mission and workflow.
- Report page accepts a natural-language civic complaint.
- Location and optional media provide additional context.
- The citizen receives a report identifier and processing status.
- My Reports provides a private history and lifecycle view.
- Public Problems exposes safe, non-sensitive problem-level information.
- About page explains what the platform does and how it works.

Production WhatsApp experience

```
Citizen → WhatsApp
"There is a water pipe leaking near the school."
↓
Twilio webhook
↓
Create report
↓
Acknowledge immediately
↓
Background processing
↓
AI understanding + media analysis
↓
Problem matching / consolidation
↓
Routing
↓
Status updates via WhatsApp
```

5. End-to-End Workflow

Phase 1 — Ingestion

Inputs may arrive through the website or WhatsApp. The ingestion layer normalizes every input into one internal report format.

- Text message → direct text input.

- Voice note → audio asset → speech-to-text.
- Photo → validated media asset → visual analysis.
- Location → captured as reported/device location with explicit provenance.
- WhatsApp metadata → sender, message ID, timestamp, media references.
- Web submission → authenticated/anonymous policy depending on the final launch configuration.

Phase 2 — Triage and Understanding

The AI pipeline turns unstructured citizen language into structured civic information.

```
Raw report
↓
Language / speech understanding
↓
Structured JSON
■■■ title
■■■ summary
■■■ category
■■■ subcategory
■■■ priority suggestion
■■■ location clues
■■■ entities
■■■ required expertise
■■■ safety flags
■■■ normalized problem fingerprint
```

Potential technologies include **Groq Whisper Large-v3** for fast transcription, an Indic-language model such as **IndicBERT** where it materially improves local-language handling, and a general multimodal/LLM layer such as **Gemini** for structured extraction and reasoning. Model choice should remain replaceable behind service interfaces.

Phase 2b — Image and Media Analysis

For images, the pipeline validates file type/size, stores media securely, and can use a vision model or YOLO-family detector where object-level detection is actually useful.

- Reject invalid or dangerous file types.
- Detect obvious non-evidence/spam patterns where possible.
- Extract useful visual cues such as road damage, garbage accumulation, damaged infrastructure, or other relevant objects.
- Never treat automated visual classification as unquestionable truth.
- Retain provenance and confidence for downstream review.

Phase 3 — Deduplication and Problem Formation

This is the heart of the platform. A report is not necessarily a unique problem.

```
Citizen reports
R1: "Handpump broken beside Ward 4 school"
R2: "Water pump not working near school"
R3: "No water from pump at Ward 4"
...
↓
Embedding generation
↓
PostgreSQL + pgvector
↓
Semantic similarity + geographic/context checks
```

↓

Relationship decision

- same problem → consolidate
- related → link / suggest
- different → keep separate

The production implementation should avoid a single blind similarity threshold. Confidence, geography, category, time, and other contextual signals should contribute to the decision.

The key data distinction

Report = one citizen's submission.

Problem = the underlying civic issue that may have many reports.

Phase 4 — Routing

Routing should be primarily deterministic and explainable, with AI assisting interpretation rather than making final jurisdiction decisions.

```
graph TD;
    Problem --> Geo[Geographic coordinates / locality];
    Geo --> PostGIS[PostGIS spatial lookup];
    PostGIS --> Jurisdiction;
    Jurisdiction --> Dept[Department / category rules];
    Dept --> Officer[Officer / organization responsibility];
    Officer --> Assignment;
```

Expertise matching can additionally connect the problem to a university, NGO, research group, engineer, or solution team.

Phase 5 — Resolution

```
graph TD
    A[Assigned] --> B[Accepted / In Progress]
    B --> C[Evidence / work updates]
    C --> D[Resolved]
    D --> E[Citizen notified]
    E --> F[Closed / reopened if necessary]
```

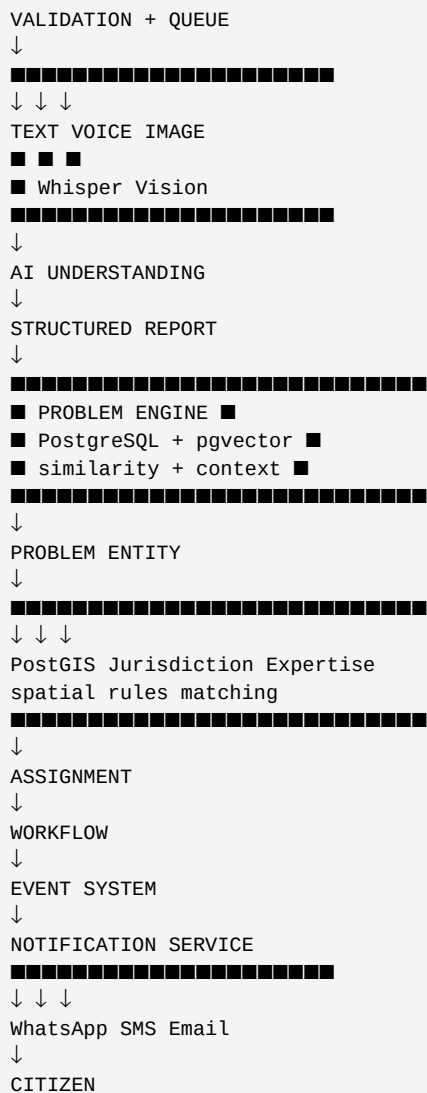
Every state transition should produce an auditable event.

6. Technical Architecture

```

graph TD
    CITIZEN --> WEB_WHATSAPP[WEB WHATSAPP]
    style CITIZEN fill:#fff,stroke:#000,stroke-width:1px
    style WEB_WHATSAPP fill:#fff,stroke:#000,stroke-width:1px
    style INGESTION_API[INGESTION API] fill:#fff,stroke:#000,stroke-width:1px
    CITIZEN --> INGESTION_API
    INGESTION_API --> WEB_WHATSAPP
  
```

The diagram illustrates the data flow from a citizen to a web WhatsApp interface. It features three main components: a 'CITIZEN' box at the top, a 'WEB WHATSAPP' box at the bottom, and an 'INGESTION API' box in the middle. A downward arrow connects the 'CITIZEN' box to the 'INGESTION API' box, and another downward arrow connects the 'INGESTION API' box to the 'WEB WHATSAPP' box, indicating a sequential flow of data.



7. Technology Stack

Layer	Planned technology	Role
Frontend	Next.js 16 + React + TypeScript	Web product and dashboards
Styling	Tailwind CSS + existing UI primitives	Minimal/classy responsive interface
Hosting	Vercel	Web deployment, server functions, environments
Auth / DB	Supabase	Authentication, PostgreSQL, storage, policies
Database	PostgreSQL	Core relational data
Vector search	pgvector	Embeddings and semantic similarity
Spatial	PostGIS	Coordinates, geographic containment and routing
WhatsApp	Twilio	Inbound/outbound messaging
Speech	Groq Whisper Large-v3 or equivalent	Voice transcription
Language	IndicBERT or equivalent where justified	Multi-language/local-language NLP
LLM	Gemini or equivalent	Structured extraction, reasoning, normalization

Layer	Planned technology	Role
Vision	YOLOv8-family / multimodal vision models	Media justification and spam/evidence checks
Queue	Production job/queue service	Async AI/media processing and retries
Monitoring	Vercel + application/error monitoring	Operational visibility
Source control	GitHub	Versioning and CI/CD

The architecture should use adapters/services so that external AI providers can be replaced without rewriting the core product.

8. Proposed Core Data Model

```

users / profiles
organizations
departments
officers

reports
■■■ submitted_by
■■■ source
■■■ text
■■■ media
■■■ reported_location
■■■ device_location
■■■ processing_status

problems
■■■ canonical title/summary
■■■ category
■■■ priority
■■■ location
■■■ confidence
■■■ status

problem_reports
■■■ problem_id
■■■ report_id
■■■ relationship_type
■■■ confidence

embeddings
■■■ entity_type
■■■ entity_id
■■■ vector

assignments
status_events
notifications
notification_deliveries
media_assets
university_interests
audit_logs

```

Why this model matters

Separating reports from problems allows 1, 10, or 10,000 citizen submissions to contribute evidence to the same underlying issue without creating 10,000 independent operational tickets.

9. API / Service Boundaries

Service	Responsibility
/api/reports	Create and retrieve citizen reports
/api/reports/[id]/process	Process/trigger report intelligence
/api/reports/[id]/duplicates	Find related reports / problem relationships
/api/problems	Public and internal problem-level operations
/api/officer/*	Officer workflow and assignments
/api/admin/*	Administrative operations
/api/university/*	University/solution-team discovery and interests
Twilio webhook	Receive WhatsApp messages/media
Notification service	Send status and operational notifications
Embedding service	Generate and persist embeddings
Routing service	Spatial/jurisdiction/expertise matching

10. AI Design Principles

- AI output must be structured and validated before entering the database.
- Every AI-generated field should have a provenance or confidence strategy where practical.
- AI should recommend; authoritative routing and permissions should be rule-based.
- Model failures must degrade gracefully instead of losing the report.
- Prompts and model versions should be versioned.
- Sensitive user data should be minimized in model requests.
- AI decisions should be auditable enough to explain why a problem was categorized, linked, or routed.
- Provider-specific code should live behind interfaces so models can be swapped.

11. pgvector Strategy

Embedding pipeline

```

Normalized problem representation
↓
Embedding model
↓
vector column in PostgreSQL
↓
indexed similarity search
↓
candidate matches
↓
contextual scoring
↓
relationship decision

```

A production duplicate score can combine semantic similarity with geographic distance, category agreement, temporal proximity, and other domain signals. The final relationship should be stored so later decisions do not need

to be reconstructed from scratch.

12. PostGIS / Spatial Strategy

- Store coordinates using a proper geographic type.
- Use spatial indexes for efficient nearby searches.
- Represent administrative boundaries/jurisdictions as geographic polygons where available.
- Match reports/problems to jurisdictions using spatial containment or nearest-area rules.
- Keep location provenance: citizen-entered, device-provided, AI-extracted, or verified.
- Never expose exact citizen coordinates publicly when they could reveal private information.

13. Twilio / WhatsApp Strategy

Twilio should be treated as a communication channel, not the core business logic.

```
WhatsApp
↓
Twilio webhook
↓
Verify webhook authenticity
↓
Normalize message
↓
Persist raw event metadata
↓
Create report
↓
Queue processing job
↓
Acknowledge citizen
↓
Process asynchronously
↓
Send lifecycle updates
```

The system should support idempotency using the provider message ID so retries do not create duplicate reports.

14. Reliability and Background Processing

Voice transcription, image analysis, embeddings, and duplicate searches should not make the user's request wait unnecessarily.

State	Meaning
received	Input persisted
queued	Background job created
processing	Pipeline currently running
processed	Pipeline completed
retrying	Recoverable failure is being retried
processing_failed	Pipeline failed and needs attention
assigned	Problem routed to responsible actor

- Assigned to responsible team

First reported
18 Sep

Last updated
19 Sep

Related reports
17

Private citizen information
NOT SHOWN

18. Officer Workspace

- Assigned problems appear in a prioritized queue.
- Officer sees canonical problem summary plus supporting citizen reports.
- Location and map context help verify responsibility.
- Officer can accept, reject/misroute, start work, update, resolve, or request more information.
- Resolution evidence can be attached.
- All transitions create status events.
- Citizen receives appropriate updates without exposing internal notes.

19. University / Solution-Team Workspace

The university layer turns civic problems into opportunities for research, engineering, design, field projects, and practical interventions.

- Discover problems matching expertise.
- Filter by category, geography, priority, and expertise.
- Review safe problem-level information.
- Submit collaboration interest.
- Admin reviews and routes the interest.
- Future versions can support project proposals, pilots, evidence, and outcome reporting.

20. Admin / Operations Workspace

- Monitor reports and problem formation.
- Review uncertain AI matches.
- Manage officers/departments.
- Manage university interests.
- Inspect processing failures.
- Inspect notification failures.
- Review audit events.

- Override or correct data with explicit reason tracking.
- Monitor system health and operational backlog.

21. Observability

Every important report should have a traceable lifecycle.

```
REPORT-123
received
↓
transcription.completed
↓
triage.completed
↓
embedding.created
↓
duplicate_candidates.found
↓
problem.created
↓
route.matched
↓
assignment.created
↓
notification.sent
↓
officer.accepted
↓
resolved
```

This lets operators answer: **Where did this report fail?** instead of guessing.

22. Failure Handling

Failure	Expected behavior
Speech service unavailable	Keep report, mark retryable, optionally request text
LLM timeout	Retry with backoff; preserve raw input
Embedding failure	Continue report processing where safe; retry vector stage
No duplicate match	Create/retain separate problem
Low-confidence match	Send to review/suggestion rather than auto-merge
Routing ambiguity	Hold for authorized review
Twilio delivery failure	Retry and record delivery status
Database transient error	Retry idempotently
Malicious upload	Reject/quarantine and log
Webhook replay	Idempotency check prevents duplicate processing

23. Product Metrics

Citizen metrics

- Reports submitted
- Successful submission rate
- Time to acknowledgement
- Time to first meaningful update
- Resolution rate
- Citizen-reported satisfaction

System metrics

- Duplicate consolidation rate
- AI processing latency
- Embedding search latency
- Routing accuracy / correction rate
- Notification delivery rate
- Retry/failure rate

Civic impact metrics

- Problems resolved
- Median resolution time
- Backlog by department
- Recurring problems
- Geographic concentration
- Number of citizens represented by consolidated problems
- University/solution-team interventions

24. Product Launch Roadmap

Stage	Goal	Major work
0. Current MVP	Prove core UX	Web reporting, auth, dashboards, lifecycle
1. Production foundation	Make it safe	RLS, authorization, validation, audit logs, privacy, error handling
2. Report → Problem	Build core intelligence	pgvector, embeddings, relationship model, confidence, review
3. WhatsApp	Open real-world ingestion	Twilio, webhooks, media, idempotency, notifications
4. Voice + vision	Handle richer input	Whisper, media pipeline, vision validation
5. Spatial routing	Automate responsibility	PostGIS, jurisdictions, departments, expertise
6. Reliability	Operate at scale	Queues, retries, observability, alerts
7. Public beta	Test with controlled users	Privacy, moderation, feedback, analytics
8. Launch	Production product	Custom domain, policies, monitoring, support, onboarding

25. Suggested Build Order

Do not implement every technology simultaneously. Build vertically so every stage produces a usable system.

```
STEP 1
Harden existing report lifecycle
↓
STEP 2
Create Problem entity + problem_reports
↓
STEP 3
Add pgvector embeddings + semantic matching
↓
STEP 4
Add confidence / review workflow
↓
STEP 5
Add Twilio WhatsApp ingestion
↓
STEP 6
Add async queue + retries
↓
STEP 7
Add Whisper voice pipeline
↓
STEP 8
Add image/vision processing
↓
STEP 9
Add PostGIS + jurisdiction routing
↓
STEP 10
Add notifications + event system
↓
STEP 11
Security/privacy/observability hardening
↓
STEP 12
Controlled public beta
↓
STEP 13
Production launch
```

26. What We Should NOT Do

- Do not automatically merge complaints solely because an embedding score is high.
- Do not let an LLM decide official jurisdiction without deterministic rules or authorized review.
- Do not expose raw citizen identity in public problem pages.
- Do not keep raw media indefinitely without a retention policy.
- Do not make citizens wait synchronously for every AI stage.
- Do not hard-code the product around one AI provider.
- Do not add AI models just to make the architecture look impressive.
- Do not launch public WhatsApp ingestion before webhook security, rate limiting, idempotency, and abuse controls exist.
- Do not treat a demo success as production reliability.

27. Example Real-World Journey

Scenario

A citizen sends a WhatsApp voice note: “There has been a water pipe leak near the school since yesterday.” They attach a photo and their phone supplies a location.

System journey

1. Twilio receives the WhatsApp message.
2. Webhook authenticity is verified.
3. Message ID is checked for duplicate delivery.
4. Audio/photo metadata is persisted.
5. Citizen receives an immediate acknowledgement.
6. Audio is transcribed.
7. Text is normalized into structured civic JSON.
8. Photo is checked for relevant visual evidence.
9. Problem embedding is generated.
10. pgvector finds nearby semantic candidates.
11. Geographic/context signals are checked.
12. A matching existing Problem is selected or a new Problem is created.
13. PostGIS determines the relevant jurisdiction.
14. Category rules identify the responsible department.
15. Assignment is created.
16. Officer receives notification.
17. Officer begins work.
18. Status update is recorded.
19. Citizen receives an update.
20. Officer submits resolution evidence.
21. Problem is marked resolved.
22. Citizen is notified.
23. Public problem status reflects the resolution.
24. Full audit trail remains available to authorized operators.

This is the central product loop: **Observe** → **Understand** → **Consolidate** → **Route** → **Act** → **Prove** → **Inform**.

28. Brand and UX Direction

The current design direction is intentionally minimal and classy rather than overloaded with dashboard-style decoration.

- Strong typography and whitespace.
- Neutral visual system with restrained accents.
- Clear hierarchy and short explanations.
- Rounded controls used selectively.
- Status represented consistently across the product.
- Public pages should feel trustworthy and civic, not like an AI experiment.
- Complexity should appear only where the user's task requires it.

The footer and About page should make authorship and purpose clear, including the team identity **The Favicons** and product attribution such as **Pratik & Team Favicons**.

29. Current Repository / Product Structure

```

src/app/
about/
admin/
api/
admin/
my-reports/
officer/
problems/
reports/
university/
login/
my-reports/
officer/
problems/
report/
signup/
university/

src/lib/
auth.ts
authenticated-fetch.ts
duplicate-score.ts
process-report.ts
report-lifecycle.ts
supabase-admin.ts

public/
favicon.ico
...

```

The existing Next.js application is already deployed on Vercel. The repository is hosted on GitHub and the Vercel project is linked. The production environment contains Supabase URL/publishable configuration and server-side secrets such as the Supabase service role and internal pipeline secret.

30. Environment / Secret Strategy

Variable	Where used	Sensitivity
NEXT_PUBLIC_SUPABASE_URL	Client/server	Public configuration
NEXT_PUBLIC_SUPABASE_PUBLIC_KEY	Client only	Public client key
NEXT_PUBLIC_APP_URL	Client/server	Configuration
SUPABASE_SERVICE_ROLE_KEY	Server only	Secret
INTERNAL_PIPELINE_SECRET	Server-to-server	Secret
TWILIO credentials	Server only	Secret
AI provider keys	Server only	Secret

No secret should be committed to GitHub. Production secrets belong in Vercel's environment configuration or an appropriate secret manager.

31. Launch Readiness Checklist

- Production domain configured.
- Supabase production project and RLS reviewed.
- All API routes authorization-tested.

- Service-role key server-only verified.
- Twilio webhook authentication implemented.
- Webhook idempotency implemented.
- Rate limits enabled.
- File validation and size limits enabled.
- Async queue and retries working.
- pgvector indexes and query performance tested.
- PostGIS routing tested against real jurisdiction data.
- AI failure fallbacks tested.
- Audit logs working.
- Notification delivery/retry tracking working.
- Privacy policy and terms prepared.
- Data retention rules implemented.
- Public/private data boundaries tested.
- Error monitoring configured.
- Backups/recovery strategy verified.
- Controlled beta completed.
- Incident/support procedure documented.

32. Definition of Done for the Product

The product is ready for a meaningful public launch when a citizen can submit a civic problem through web or WhatsApp and the platform can reliably carry that problem from ingestion through understanding, consolidation, routing, action, resolution, and notification—while preserving security, privacy, explainability, and operational recovery.

A visually polished interface is important, but the defining product quality is **trustworthy closed-loop civic action**.

33. Final Product Vision

The long-term vision is not simply a complaint portal. It is a shared infrastructure layer between citizens, public authorities, institutions, and solution teams.

A citizen should not need to understand government structure, department boundaries, AI, or databases. They should be able to describe a real problem in ordinary language.

The platform should absorb the complexity: understand the report, determine whether others have seen the same problem, construct the underlying problem, identify who is responsible, connect relevant expertise, track action, and communicate progress.

In that sense, the product's ultimate unit of value is not the number of reports submitted. It is the number of **real civic problems moved from observation to verified action**.

The product sentence

“Tell us what is wrong. We turn it into a problem the right people can act on.”

The architecture sentence

“AI understands; PostgreSQL remembers; pgvector connects; PostGIS locates; rules route; people act; Twilio keeps citizens informed.”

The operating principle

Every report should either become part of a meaningful problem, become evidence for an existing problem, or be safely rejected—with no silent disappearance.